

Smart Contract Security Audit

DeliveryEscrow - USDC delivery-marketplace escrow on Base

Target	0x642da3859deD225Cf42efd21346e317e8e26F58e
Network	Base mainnet (chain ID 8453)
Contract name	DeliveryEscrow (BaseScan: Source Code Verified, Exact Match)
Compiler	Solidity v0.8.30+commit.73712a01, optimizer on (10,000 runs), EVM version osaka, MIT
Deployer	0x445878e492A53cfE567FB89C805b3316F6061b35 (also deployed the earlier contract 0xb578...3990B)
Review type	Manual review of ABI, constructor arguments, compiler settings and EVM bytecode. The Solidity source text could not be retrieved in this session (see Section 2).
Report date	25 September 2026
Overall risk	MODERATE - no Critical or High issue identified; 3 Medium, 3 Low, 5 Informational

This report is a best-effort technical review. It does not guarantee the absence of vulnerabilities and does not replace a formal audit by a dedicated security firm. Findings are labelled with a confidence level because part of the analysis is bytecode-level.

1. Executive summary

DeliveryEscrow is a marketplace escrow denominated in native USDC on Base. Sellers list products, buyers create orders that lock the product price plus delivery fees, couriers bid for the delivery job and post bonds, and the parties confirm pickup and delivery. A return flow (return courier, return bonds, return deadlines) is included. Settlement is pull-based: functions credit an internal balance and each beneficiary later calls `claim()`.

At the time of the snapshot the contract was about 26 minutes old with 0 transactions and 0 balance, so **no user funds are currently at risk**. No external audit has been submitted on BaseScan.

Overall assessment: moderate risk. No path was found by which the admin address, a single user, or an outside attacker can directly take user funds. The main concerns are a front-running window on order creation, the removal of the value caps that the earlier deployment had, and settlement rules that depend heavily on deadlines and party liveness.

ID	Severity	Title	Confidence
M-01	Medium	createOrder is not bound to the terms the buyer saw (price/metadata can change before inclusion)	High (ABI); Medium (exploitability)
M-02	Medium	No per-order or global value cap on a new, unaudited contract	High
M-03	Medium	Deadline- and liveness-dependent settlement; no arbiter (design risk to validate)	Low-Medium
L-01	Low	Guardian and treasury are the same immutable address; no rotation	High
L-02	Low	Flat claim fee absorbs small balances (dust forfeiture)	High
L-03	Low	Order does not snapshot the product metadata hash	Medium
I-01	Info	Full dependence on native USDC admin powers (blacklist / pause / upgrade)	High
I-02	Info	Immutable configuration is irreversible (decoded values look sane)	High
I-03	Info	Timestamp windows on an L2 sequencer	High
I-04	Info	High state-machine complexity (15 order states, bit-packed storage)	High
I-05	Info	Source text not reviewed; findings marked Medium/Low confidence need source confirmation	n/a

Changes compared with the earlier deployment (0xb578...3990B). Both contracts share the same logic family (identical function selectors except three). This version **removes the third-party claim function** (fixes a fee-griefing vector) and **removes the 5,000 USDC global cap and 100 USDC per-order cap** (fixes cap-filling DoS but removes an important exposure limit). Fees, periods and the penalty are now constructor immutables instead of hard-coded constants. Details in Section 8.

2. Scope, method and limitations

Material reviewed (from the BaseScan page of the target address):

- Full contract ABI (functions, events, custom errors) and compiler settings.
- Complete creation bytecode, including the ABI-encoded constructor arguments appended at the end.
- Runtime bytecode (the dispatcher and function bodies), traced manually for access control, state transitions, token flows and accounting.

Method: selector-to-ABI mapping; decoding of constructor arguments and inlined constants; tracing of the pause check, role checks, deposit path, credit path, claim path, fee withdrawal and settlement helpers; storage-layout reconstruction; comparison against the earlier contract by selector set and constants; pattern comparison against OpenZeppelin v5 idioms (ReentrancyGuard, SafeERC20, SafeCast, Math.mulDiv).

Limitations:

- **Source text was not available.** BaseScan marks the contract as verified, but the Source tab is rendered by JavaScript and was not returned in this session. Function-level conclusions therefore come from the ABI plus bytecode, not from reading Solidity.
- No dynamic testing, fuzzing, fork simulation or automated static analysis was run.
- Order-state names are not in the ABI (the enum has 15 values). The lifecycle in Section 5 is inferred from function names, errors and events.
- 'Verified - Exact Match' on BaseScan means the deployed bytecode matches the submitted source. It does not mean the contract is safe.

Confidence levels: High = directly visible in ABI or traced in bytecode. Medium = consistent with ABI and partial tracing. Low = inferred from names and design patterns; needs source confirmation.

3. Contract snapshot

Property	Observation
Address	0x642da3859deD225Cf42efd21346e317e8e26F58e
Creator / age	0x445878e492A53cfE567FB89C805b3316F6061b35, created about 26 minutes before the snapshot (24 Sep 2026, ~21:57 UTC)
Activity	0 transactions, 0 ETH, no token balances, no audit submitted
Proxy / upgradeability	None observed. No DELEGATECALL, CALLCODE or SELFDESTRUCT pattern in the reviewed code.
ETH handling	All functions are non-payable or view; no receive/fallback in the ABI. ETH cannot be deposited.
External calls	Only to the payment token (STATICCALL for balanceOf/decimals, CALL for transfer/transferFrom).
Libraries	OpenZeppelin-v5-style ReentrancyGuard (ERC-7201 slot 0x9b779b17...5f00), SafeERC20 (SafeERC20FailedOperation), SafeCast (SafeCastOverflowedUintDowncast), Math.mulDiv for basis-point maths.
Payment token	0x833589fCD6eDb6E08f4c7C32d4f71b54bdA02913 (native USDC on Base), immutable

4. Deployment configuration (decoded constructor arguments)

The 12 constructor arguments were decoded from the tail of the creation bytecode. All are immutables baked into the runtime code, so none can be changed after deployment.

Argument	Raw value	Meaning
paymentToken_	0x833589fC...A02913	Native USDC on Base
treasury_	0xf4EDaEE3C9CaCC28e9E2EbcBf60962a8E405992A	Receives accrued fees (withdrawFees)
guardian_	0xf4EDaEE3C9CaCC28e9E2EbcBf60962a8E405992A	Can pause new activity. Same address as treasury.
listingFee_	200,000	0.20 USDC per product listing

Argument	Raw value	Meaning
purchaseFee_	1,000,000	1.00 USDC per order created
actionFee_	200,000	0.20 USDC for product updates/status actions, and the flat fee applied per claim
courierPenaltyBps_	1,500	15% - courier bond = ceil(price x 15%)
inspectionPeriod_	259,200 s	3 days
buyerAbsentGracePeriod_	86,400 s	1 day
returnAcceptPeriod_	259,200 s	3 days
returnPickupPeriod_	259,200 s	3 days
returnDeliveryPeriod_	2,592,000 s	30 days

Constructor sanity checks observed: token, treasury and guardian are non-zero and not the contract itself; `paymentToken.decimals()` == 6; penalty <= 5,000 bps (MAX_COURIER_PENALTY_BPS); each of the five periods is non-zero and <= 365 days. No upper bound on the three fee values was identified (they are immutable, so this only matters at deployment).

Fixed constants: BPS_DENOMINATOR = 10,000; MAX_COURIER_PENALTY_BPS = 5,000; COURIER_OFFER_FEE = 50,000 (0.05 USDC); MAX_ORDER_HORIZON = 31,536,000 s (365 days); plus fixed courier-selection, funding and return-matching windows in the 1-day to 30-day range.

5. Architecture and trust model

Actors

- **Seller** - lists and updates products, posts a seller bond, confirms handovers.
- **Buyer** - creates the order (locks price + forward and return delivery fees), approves the courier, confirms delivery or reports a mismatch, can request a return.
- **Courier / return courier** - submits an offer (0.05 USDC fee), is approved by the buyer, funds a bond, confirms pickup and delivery. Cannot be the buyer or the seller (checked in acceptCourier).
- **Guardian / treasury** - one immutable address. Guardian: pause/unpause new activity. Treasury: withdraw accrued fees.

Lifecycle (inferred from function names, errors, events)

- Listing: listProduct(price, metadataHash) -> updateProduct / deactivateProduct / reactivateProduct.
- Order: createOrder(productId, forwardFee, returnFee, deliveryMode) -> courier offers (acceptCourier) -> buyer approves (approveCourier) -> fundCourierBond, fundSellerBond.
- Delivery: confirmPickup, confirmDelivery, confirmMismatch, confirmSafeDrop; reportBuyerAbsent / beginBuyerAbsentReturn; finalizeInspection; finalizeCourierReportedDelivery.
- Return: requestReturn -> acceptReturnCourier / approveReturnCourier -> fundReturnCourierBond -> confirmReturnPickup / confirmReturnDelivery -> finalizeCourierReportedReturn.
- Failure and expiry: cancelUnmatchedOrder, expireOrderBeforePickup, expireUnmatchedReturn, declareCourierDefault.
- Payout: internal credits; beneficiary calls claim().

Storage layout (reconstructed)

Slot	Content
0 / 1	nextProductId / nextOrderId (both start at 1)
2	totalLiability - all funds owed to users (escrow, bonds, unclaimed credits)
3	accruedFees - protocol fees not yet withdrawn
4	newActivityPaused flag
5	products[id]: seller, price (uint128), metadataHash, status
6	orders[id]: 4 role addresses, uint128 amounts (price, fees, escrow, 3 bonds), uint64 deadlines, uint8 confirmation counters and state
7 / 8	courierOffers[orderId][courier] (periods + active flag) / returnCourierOffers[orderId][courier] (bool)
9	credits[account]

Access-control matrix

Function group	Who can call	Blocked by pause?
listProduct, updateProduct, reactivateProduct	Any address (updates: product seller only)	Yes
createOrder	Any address except the product seller	Yes
deactivateProduct	Product seller	No
acceptCourier / return offers	Any address except the buyer and the seller	No
approveCourier / approveReturnCourier	Buyer	No
Bond funding, pickup/delivery/mismatch confirmations	Designated role for the order	No
Expiry / finalize / default functions	Several are permissionless once their deadline has passed	No
claim()	Beneficiary only (msg.sender)	No
setNewActivityPaused	Guardian	n/a
withdrawFees(recipient, amount)	Treasury; amount <= accruedFees	No

Key trust conclusion. Pause only stops *new* activity (listing, product updates, reactivation, order creation). Exits - cancellation, settlement, timeouts and claim() - are not pausable, so the guardian cannot lock funds already in the system. The treasury can only withdraw *accrued fees*, and the solvency invariant is re-checked after the transfer. There is no owner, no upgrade path and no arbiter role.

6. Security controls observed

Control	Evidence	Assessment
Reentrancy protection	nonReentrant wrapper present on every state-changing entry point, including claim() and withdrawFees()	Good
Pull-payment settlement	Settlement functions call an internal credit helper that increments credits[account] and emits FundsCredited; only claim() and withdrawFees() move tokens out	Good. A blacklisted recipient cannot block anyone else's settlement
Checks-effects-interactions in claim()	credits[msg.sender] is zeroed and totalLiability reduced before the token transfer	Good
Exact-amount deposits	Deposits read balanceOf before/after transferFrom and revert (UnsupportedTransferBehavior) if the delta differs from the amount	Good. Guards against fee-on-transfer or short-transfer tokens
Safe token calls	SafeERC20-style handling of return data (SafeERC20FailedOperation)	Good
Solvency invariant	After claim() and withdrawFees(): USDC balance >= totalLiability + accruedFees, else Insolvent. View: accountingInvariantHolds()	Good. Uses >=, so a donation cannot brick the contract
Checked narrowing casts	uint128 / uint64 packing uses SafeCast; overflow reverts	Good
State guards	Every transition validates the current order state and reverts with InvalidState(expected, actual)	Good
Role separation	Buyer != seller in createOrder; courier != buyer/seller in acceptCourier	Good
Minimal admin powers	Guardian: pause new activity. Treasury: fee withdrawal only. No parameter setters.	Good

7. Findings

M-01 createOrder is not bound to the terms the buyer saw		Medium
Confidence	High that the ABI has no expected-terms parameters; Medium on real-world exploitability	
Location	createOrder(productId, forwardDeliveryFee, returnDeliveryFee, deliveryMode); updateProduct(productId, price, metadataHash)	
Description	createOrder takes no expected price and no expected metadata hash. At execution it reads the product's current price from storage and pulls price + delivery fees from the buyer. updateProduct lets the seller change price and metadataHash at any time for a flat 0.20 USDC, with no delay or cooldown. Between the moment a buyer signs and the moment the transaction is included (or by front-running it), a seller can raise the price or swap the metadata the buyer reviewed.	
Impact	A buyer can be charged more than they agreed, or buy an item whose description hash changed. The only ceiling is the buyer's USDC allowance to the contract (often unlimited) - and, unlike the earlier deployment, there is no 100 USDC per-order cap (see M-02).	
Recommendation	Add expectedPrice (or maxTotal) and expectedMetadataHash parameters to createOrder and revert on mismatch; or keep a productVersion counter incremented by updateProduct and require the buyer to pass it. Optionally add a short delay before a price increase takes effect. Add a fork test that calls updateProduct between quote and createOrder.	

M-02 No per-order or global value cap on a new, unaudited contract		Medium
Confidence	High	
Location	createOrder, listProduct, updateProduct, fee/credit accounting	
Description	The earlier deployment enforced a 100 USDC per-order limit and a 5,000 USDC global limit on locked liabilities. In this version the cap getters (selectors 0x65b60c0d and 0x3e3bc77e) are gone and no comparison against those limits remains in the deposit paths. The contract is minutes old, has no audit submitted and has a large state machine (finding I-04).	
Impact	Any undiscovered logic or accounting bug now has an unbounded blast radius: exposure is limited only by how much users deposit.	
Recommendation	Reintroduce caps as constructor immutables (per-order and global), or a guardian-controlled cap that can only be lowered. Raise them in stages after audit and monitoring. Alert on accountingInvariantHolds() == false.	

M-03 Deadline- and liveness-dependent settlement; no arbiter		Medium
Confidence	Low-Medium (inferred from ABI names, errors and events; needs source confirmation)	
Location	finalizeInspection, finalizeCourierReportedDelivery, finalizeCourierReportedReturn, reportBuyerAbsent, beginBuyerAbsentReturn, declareCourierDefault, expire* functions	
Description	Outcomes are decided by role confirmations plus timeouts (3-day inspection, 1-day buyer-absent grace, 3-day return windows, 30-day return delivery). Several finalize/expiry functions are permissionless after a deadline. There is no arbitrator for conflicting claims (ConflictingConfirmation, mismatch vs delivery). A party that is offline during its window may lose the right to dispute, and the default outcome then executes.	
Impact	Possible loss of funds by an inattentive buyer or courier (for example a courier-reported delivery that finalizes after the inspection window), or deadlock/griefing when parties disagree. Severity depends on which default each timeout path picks - this must be checked in source.	
Recommendation	Build a full table of state x actor x time -> outcome and verify that every timeout defaults to the least harmful result for the party that can still be offline. Provide off-chain reminders. Consider a longer minimum inspection window and an on-chain 'extend window' action. Model collusion cases (seller with a second-wallet courier who still needs buyer approval).	

L-01 Guardian and treasury are the same immutable address		Low
Confidence	High	
Location	constructor (guardian_, treasury_), setNewActivityPaused, withdrawFees	
Description	Both roles are set to 0xf4EDaEE3C9CaCC28e9E2EbcBf60962a8E405992A, which is not the deployer. Roles cannot be rotated or split. If that key is lost or compromised, an attacker can pause new activity permanently (marketplace freeze, exits still work) and sweep accrued fees; if the key is lost while paused, new activity stays stopped.	
Impact	Availability and fee-theft risk only; escrowed user funds are not reachable through these roles.	
Recommendation	Confirm that the address is a multisig (ideally with a timelock). If redeploying, use separate guardian and treasury addresses and consider a two-step, delayed role transfer.	

L-02 Flat claim fee absorbs small balances		Low
Confidence	High	
Location	claim()	
Description	claim() charges fee = min(credit, actionFee) where actionFee is 0.20 USDC. Any account whose total credit is <= 0.20 USDC receives nothing and forfeits the whole balance to the treasury; larger balances pay a fixed 0.20 USDC each time they claim. Credits from several orders accumulate per account, so batching mitigates the cost.	
Impact	Small delivery fees or bond refunds can be fully consumed by the claim fee. No effect on solvency.	
Recommendation	Document the behaviour in the UI, or make the fee proportional with a floor that cannot exceed a fixed share of the balance, or let users skip the fee below a threshold.	

L-03 Order does not snapshot the product metadata hash		Low
Confidence	Medium	
Location	Order struct, updateProduct	
Description	The order stores productId and the price at creation, but the ABI shows no metadataHash field in Order. After an order exists, the seller can still call updateProduct and change the product's metadataHash. On-chain, the record of what was ordered then depends on the ProductUpdated event history.	
Impact	Weakens dispute evidence for mismatch cases.	
Recommendation	Store metadataHash in the order at creation (or block metadata updates while the product has open orders).	

Informational

- **I-01 USDC dependency.** Native USDC on Base is upgradeable and can blacklist or pause. If the contract address is blacklisted, all funds freeze. If a single beneficiary is blacklisted, only that account's claim() fails (pull-payments isolate it). Disclose this to users.
- **I-02 Irreversible configuration.** Fees, periods and penalty are immutable. The decoded values (0.20 / 1.00 / 0.20 USDC, 15%, 1-3-30 day windows) look consistent with the earlier deployment. A wrong value would require redeployment.
- **I-03 Timestamps.** Deadlines use block.timestamp on an L2 where the sequencer sets time. Drift is negligible against 1-day and longer windows. Casts to uint64 are checked. The evmVersion 'osaka' setting is unusual: no post-Cancun opcodes were noticed in the reviewed code (PUSH0 is used), and deployment succeeded, but confirm your toolchain and target-chain support.
- **I-04 Complexity.** About 60 external functions, 15 order states, four roles per order and hand-packed 32-byte slots. The main risk class is state desynchronisation. Invest in invariant/fuzz testing (Section 9).
- **I-05 Source-level review pending.** Findings M-03 and L-03 in particular should be re-checked against the Solidity source.

8. Comparison with the earlier deployment (0xb578...3990B)

The two contracts share almost the same selector set, so they belong to the same code family. The earlier report described the earlier contract with generic terms ('offers', 'deals'); with this ABI those map to products and orders.

Aspect	Earlier (0xb578...)	This contract (0x642d...)	Effect
Source	Unverified	Verified, exact match, ABI public	Better
Configuration	Fees, periods, penalty hard-coded	Constructor immutables (same values)	Neutral
Third-party claim (selector 0xddeae033)	Present: anyone could trigger someone else's claim and force the flat fee	Removed; claim() is beneficiary-only	Fixes earlier L-01
Per-order cap (100 USDC) and global cap (5,000 USDC)	Enforced	Removed	Fixes cap-filling DoS, but see M-02
Token	USDC address hard-coded	Constructor argument, decimals()==6 check, USDC used	Neutral
Guardian / treasury	One hard-coded address	Two constructor args, same address	Same weakness (L-01)
Price change before order (TOCTOU)	Present	Still present	Open (M-01)

9. Recommendations and test plan

Before real funds (priority P0)

- Fix M-01 (bind createOrder to expected price/metadata or a version).
- Decide on caps (M-02) and set conservative launch limits.
- Confirm the guardian/treasury address is a multisig (L-01).
- Publish the audit status honestly to users; no audit is submitted yet.

Before scaling (P1)

- Commission a source-level audit; supply this report's open items (M-03, L-03) to the auditors.
- Write Foundry invariant tests for the properties below and run them with realistic actor handlers (seller, buyer, courier, return courier, keeper, guardian).
- Fork tests against real Base USDC, including a blacklisted recipient and a blacklisted contract address.
- Run Slither/Aderyn and consider formal verification (Certora/Halmos) for the state machine and accounting.

Invariants to test

#	Invariant
1	USDC.balanceOf(this) >= totalLiability + accruedFees after any call sequence (accountingInvariantHolds() must stay true)
2	totalLiability == sum(credits) + sum(orderLiability(id)) over open orders
3	Terminal order states are absorbing: no escrow or bond field is non-zero afterwards
4	Value conservation per order: total credited to all parties + fees == total locked for that order; each amount is released exactly once
5	Role exclusivity: buyer != seller; courier and return courier are never buyer or seller
6	No credit is ever assigned to address(0)
7	Pause never blocks cancel, expire, finalize or claim paths
8	Interleaving updateProduct / createOrder cannot make a buyer pay more than the terms they specified (after M-01 fix)
9	Deadlines only move forward and each window is enforced with consistent inclusive/exclusive comparisons

Appendix A - ABI surface

Category	Functions
Product management	listProduct, updateProduct, deactivateProduct, reactivateProduct, getProduct, nextProductId
Order creation and views	createOrder, getOrder, nextOrderId, orderLiability, requiredCourierBond
Courier matching	acceptCourier, approveCourier, fundCourierBond, fundSellerBond, getCourierOffer
Delivery confirmations	confirmPickup, confirmDelivery, confirmMismatch, confirmSafeDrop, reportBuyerAbsent, beginBuyerAbsentReturn, finalizeInspection, finalizeCourierReportedDelivery
Returns	requestReturn, acceptReturnCourier, approveReturnCourier, fundReturnCourierBond, confirmReturnPickup, confirmReturnDelivery, finalizeCourierReportedReturn, hasReturnCourierOffer, expireUnmatchedReturn
Expiry and failure	cancelUnmatchedOrder, expireOrderBeforePickup, declareCourierDefault
Funds and fees	claim, claimable, withdrawFees, accruedFees, totalLiability, accountingInvariantHolds
Admin	setNewActivityPaused, newActivityPaused, guardian, treasury
Configuration getters	paymentToken, listingFee, purchaseFee, actionFee, courierPenaltyBps, inspectionPeriod, buyerAbsentGracePeriod, returnAcceptPeriod, returnPickupPeriod, returnDeliveryPeriod, BPS_DENOMINATOR, MAX_COURIER_PENALTY_BPS, MAX_ORDER_HORIZON, MAX_RETURN_MATCHING_PERIOD, COURIER_FUNDING_PERIOD, COURIER_SELECTION_PERIOD, COURIER_OFFER_FEE, SELLER_FUNDING_PERIOD

Appendix B - Custom errors

Error	Meaning (inferred)
Unauthorized, ZeroAddress, InvalidAmount, InvalidConfiguration	Access, input and constructor validation
InvalidState(expected, actual), OfferNotFound	Wrong order/product state or missing courier offer
DeadlinePassed, DeadlineNotReached, InvalidDeadline	Time-window enforcement
AlreadyConfirmed, ConflictingConfirmation, CourierAlreadyReportedDelivery	Confirmation logic
NewActivityPaused	Pause gate on new activity
NothingToClaim, Insolvent, UnsupportedTransferBehavior	Payout, accounting and token-behaviour guards
ReentrancyGuardReentrantCall, SafeERC20FailedOperation, SafeCastOverflowedUintDowncast	OpenZeppelin library errors

Disclaimer. This document is a technical review based on the ABI and bytecode available on BaseScan and does not include the Solidity source text, dynamic testing or automated tooling. It is not investment, legal or security certification and does not guarantee that the contract is free of vulnerabilities. Before handling real funds, obtain a full source-level audit.